ApproxPilot-SE: Self-Evolving Design Space Exploration for Approximate Accelerators

Yanzhe Wang, Cheng Liu, Fangfa Fu, Bing Yang and Jinxiang Wang

***Abstract*—Approximate computing improves accelerator power, performance, and area (PPA) by replacing various arithmetic units with the most appropriate approximate implementations while preserving application-level output quality. However, exploring the resulting combinatorial design space requires costly evaluations of the resulting hardware designs. Surrogate models enable efficient design space exploration (DSE) by predicting hardware cost and application quality without evaluating every candidate. Building accurate surrogates, however, often requires substantial labeled data for pretraining or training a separate model for each target accelerator. Acquiring these data incurs considerable evaluation time, even when performed offline. This paper presents ApproxPilot-SE, a self-evolving DSE framework that begins exploration with surrogate models trained on a limited initial dataset. To support search before the models become sufficiently accurate, ApproxPilot-SE retrieves similar, previously measured configurations and uses their prediction residuals to calibrate latency estimates. During DSE, promising configurations on the predicted Pareto front are selected for logic synthesis and application-quality evaluation. These measurements verify the hardware cost and output quality of important candidate designs, reducing reliance on potentially biased surrogate predictions. The evaluated configurations are then incorporated into the training dataset and calibration reference set to refine subsequent predictions. This feedback enables DSE and surrogate learning to co-evolve, progressively improving the exploration of approximate accelerator designs. Across five accelerator benchmarks and five structural similarity index (SSIM) retention requirements, ApproxPilot-SE achieves the highest or jointly highest verified PPA hypervolume gain in 24 of 25 settings with 720 training configurations per method. At 90% SSIM retention, it outperforms the stronger baseline on every benchmark, with a geometric-mean hypervolume-gain ratio of 3.05. These results demonstrate improved verified hardware tradeoffs across different accelerator structures and application-quality requirements.**

***Index Terms*—approximate accelerators, graph neural networks, surrogate modeling, design space exploration, latency prediction.**

I. Introduction

APPROXIMATE computing exploits application error tolerance to improve hardware power, performance, and area (PPA) while preserving acceptable output quality [1]–[3]. For a domain-specific accelerator, these tradeoffs can be realized by replacing arithmetic units with appropriate approximate implementations drawn from a component library [4]. The design decisions include which arithmetic sites should remain exact, which should be approximated, and which implementation should be assigned to each approximated site. These choices are coupled: local errors propagate through the computation, while logic synthesis and interactions along circuit paths determine the hardware cost of the complete design. Consequently, selecting units solely from their individual error and cost characteristics does not establish the best accelerator-level PPA–quality tradeoff.

The number of possible assignments grows combinatorially with the number of configurable sites and available implementations. In our evaluation, DCT and Laplace contain 76 and 60 configurable sites, with initial design spaces of approximately 3.26×10^{91} and 1.77×10^{72} configurations, respectively. Exploring such spaces requires distinguishing promising designs without synthesizing and executing every candidate. Yet the PPA and application quality needed to make these decisions are themselves expensive to obtain: logic synthesis evaluates hardware cost, and application execution measures the quality impact of the selected substitutions. Efficient design space exploration (DSE) therefore depends on how these evaluations are allocated.

Surrogate models address this challenge by predicting hardware cost and application quality for unevaluated configurations. AutoAx [5] uses learned predictors to support automated exploration of approximate-component assignments. ApproxPilot [6] employs graph neural networks (GNNs) to model circuit topology and incorporates critical-path information into delay prediction. ApproxGNN [7] learns reusable approximate-component representations and demonstrates transfer across related accelerator structures. These approaches enable many candidate configurations to be explored through inexpensive predictions rather than repeated hardware evaluations. However, obtaining accurate predictors can require substantial labeled data, collected either to train models for each target accelerator or to build a reusable pretrained model. Although pretraining costs can be amortized across targets, constructing the underlying dataset still requires expensive evaluations; target-specific training likewise incurs data-acquisition costs for the accelerator being optimized. Performing this work offline does not remove its contribution to the overall design effort.

This motivates a different use of the evaluation budget: begin DSE with a limited labeled dataset and improve the models using measurements selected during exploration. The challenge is that early models may contain prediction bias that distorts candidate rankings and the apparent Pareto front. A promising predicted configuration may fail to deliver the expected PPA or output quality when evaluated, and repeatedly

Yanzhe Wang, Fangfa Fu, and Jinxiang Wang are with Harbin Institute of Technology, China.

Cheng Liu is with North China Electric Power University, China.

Bing Yang is with Harbin University of Science and Technology, China.

trusting such predictions can direct search toward misleading tradeoffs. The framework must therefore support useful exploration before the surrogates are broadly accurate and obtain feedback that is relevant to subsequent optimization.

We present **ApproxPilot-SE**, a self-evolving DSE framework that combines measured-reference latency calibration with Pareto-guided evaluation and iterative surrogate training. Starting from a limited set of measured configurations, configuration-aware GNNs predict accelerator PPA and application quality. During search, the framework retrieves similar measured configurations and uses their prediction residuals to calibrate latency estimates. This local correction anchors predictions to available measurements and can cancel a shared prediction-bias component when nearby configurations exhibit similar errors. A path-aware readout supports the underlying latency model by preserving directed circuit dependencies without requiring explicit critical-path labels.

During DSE, Pareto-relevant candidates are selected for logic synthesis and application-quality evaluation. These measurements establish the actual hardware cost and output quality of important candidate designs, reducing reliance on their predicted performance. The evaluated configurations expand both the surrogate training dataset and the latency-calibration reference set. The updated models and references then guide the next search round. In this closed loop, search determines where new measurements are acquired, and the measurements reshape the predictions used by subsequent search. We use *self-evolving* to describe this feedback-driven adaptation of the complete DSE framework; its mechanism is the co-evolution of surrogate learning and design exploration.

The main contributions of this work are summarized as follows:

- **A self-evolving framework for approximate accelerator DSE.** ApproxPilot-SE couples multiobjective search with Pareto-guided acquisition of measured configurations and iterative surrogate retraining. This process directs new training data toward optimization-relevant regions instead of relying exclusively on a model trained before exploration.
- **Measured-reference local latency calibration.** ApproxPilot-SE retrieves nearby evaluated configurations and uses their prediction residuals to correct latency estimates without an additional learned correction model. Newly acquired measurements improve both the surrogate training set and the calibration reference set, while a path-aware readout provides structural support for latency prediction.
- **Ground-truth-verified evaluation across five accelerators.** With 720 training configurations per method, ApproxPilot-SE achieves the highest or jointly highest PPA hypervolume gain in 24 of 25 benchmark–quality settings. At 90% structural similarity index (SSIM) retention, the geometric mean of its HV Gain ratios relative to the stronger baseline on each benchmark is 3.05.

II. Background and Related Work

A. Automated Design of Approximate Accelerators

The automated design of approximate accelerators involves two closely related tasks: developing arithmetic units with different error characteristics and hardware costs, and determining how to combine these units within a complete accelerator. EvoApprox8b [4] targets ASIC implementations and contains 430 nondominated 8-bit approximate adders and 471 nondominated 8-bit approximate multipliers. SMApproxLib [8], in contrast, targets FPGAs and is the first open-source library of FPGA-based approximate multipliers. Rather than selecting components from a predefined library, ABACUS [9] applies approximation transformations directly to behavioral hardware descriptions to identify Pareto-optimal designs in the accuracy–power tradeoff. These approaches define searchable design spaces through either component selection or design transformation.

Candidate evaluation is a key factor in determining how many configurations can be explored. AutoAx [5] trains system-level prediction models using the hardware and error characteristics of arithmetic units. It evaluates configurations through these surrogates during combinatorial search and performs ground-truth evaluations of the shortlisted candidates. AxHLS [10] integrates approximate functional units with high-level synthesis (HLS), using analytical models to estimate resource requirements and design accuracy. For FPGAs, ApproxFPGAs [11] investigates the adaptation and exploration of approximate arithmetic components originally developed for ASICs, using machine learning to reduce the time required to identify an optimal set of approximate components for FPGA implementation. Xel-FPGAs [12] adopts a multistage design flow in which basic arithmetic components are composed into approximate functional modules, which are subsequently used for system-level optimization.

B. Hardware Performance and Output Quality Modeling

System-level performance prediction requires relating local operation characteristics to the dependencies within a design. Graph representations provide a natural basis for modeling these relationships. Wu et al. [13] represent C/C++ programs as graphs, introduce a benchmark suite of 40,000 synthesizable C programs, and develop a hierarchical GNN for hardware performance prediction. SNS [14] predicts area, power, and timing from RTL designs. It extracts a graph intermediate representation (GraphIR) from Verilog, uses a lightweight Transformer model, Circuitformer, to predict path-level PPA, and aggregates these predictions into circuit-level estimates through a simple multilayer perceptron (MLP). By extending model inputs from aggregate statistics to structured design representations, these methods enable models to learn how local operations relate to the overall hardware implementation.

The organization of structural information should also reflect the prediction target. PowerGear [15] incorporates activity information and connectivity between operations into a heterogeneous edge-centric graph model. Its edge-centric neighborhood aggregation captures edge semantics and structural properties for power estimation in HLS-based FPGA designs.

TABLE I
COMPARISON OF REPRESENTATIVE SURROGATE-ASSISTED HARDWARE DESIGN SPACE EXPLORATION APPROACHES.

Feature	AutoAx	ApproxPilot	ApproxGNN	ARS-Flow 2.0	**ApproxPilot-SE (Ours)**
Surrogate model	RF	GNN	GNN	PSOGPR	**GNN**
Hardware cost prediction	✓	✓	✓	✓	✓
Approximation quality prediction	✓	✓	✓	N/A	✓
Circuit-graph modeling	✗	✓	✓	✗	✓
Gate-level component embeddings	✗	✗	✓	✗	✗
Path modeling	✗	Critical-path node prediction	✗	✗	**Path-aware aggregation**
Prediction calibration	✗	✗	Global offset	✗	**Local reference calibration**
Search-time surrogate updates	✗	✗	✗	✓	✓

For delay modeling, the mapping of operations to hardware resources and the order of signal propagation are particularly important. Ustun et al. [16] use GNNs to identify operation-to-resource mapping patterns on FPGAs to support operation delay prediction. Guo et al. [17] introduce TimingPredict for pre-routing timing prediction, using edge-delay prediction as a local auxiliary task to improve model performance. Compared with generic graph-level aggregation, these methods incorporate target-specific hardware dependencies into the modeling process.

Approximate accelerators also require models of how arithmetic errors affect application outputs. Mo et al. [18] study the relationship between the error characteristics of approximate multipliers and accuracy loss in neural network inference. They rank statistical error features by importance and use these features to predict the inference accuracy loss induced by approximate multipliers.

Hardware cost features and numerical error features serve different modeling purposes. Our prior work, ApproxPilot [6], combines structural modeling with error characterization. It represents arithmetic units as nodes and their connections as edges, encoding unit-level PPA, operation types, and error metrics as node features. To capture the path dependence of delay, ApproxPilot adopts a two-stage model. The first stage performs node-level classification to identify arithmetic nodes on the critical path. The second stage augments the original node features with the classification results and performs graph-level regression to predict accelerator PPA and output quality. The critical-path node labels are obtained from synthesis reports. Together, these studies demonstrate the value of circuit structure for fast performance estimation and highlight the importance of path information in delay modeling.

C. Surrogate Model Adaptation and Prediction Calibration

Model adaptation seeks to transfer learned representations and predictive mappings to new target designs. ApproxGNN [7] learns transferable component representations by jointly training a component embedding network and a system-level prediction network. The embedding network processes gate-level Verilog descriptions of approximate arithmetic units to generate feature vectors. The prediction network operates on the accelerator graph, using component embeddings and structural information to predict quality of results (QoR) and hardware cost. For each training batch, the embedding network first processes the entire component library, and the resulting embeddings are inserted into the accelerator graph as node features. Gradients propagate through both networks, allowing the embedding network to be trained despite the limited number of distinct components in the library.

In its transfer experiments, ApproxGNN constructs a source-domain dataset, denoted 16K, from randomly generated 3×3 convolution accelerators. The trained models are then transferred to the Small Gaussian and Large Gaussian filters, neither of which is included in the source-domain dataset. To mitigate systematic underestimation of PSNR after transfer, a uniform additive offset is estimated from a small set of randomly selected target samples. This correction leaves pairwise prediction differences and candidate rankings unchanged. ApproxGNN also investigates target-domain fine-tuning, either updating only the system-level prediction network or jointly updating the component embedding and prediction networks [7].

D. Design Space Exploration

In surrogate-assisted DSE, the training dataset can be updated as the search progresses. ParEGO [19] addresses multi-objective optimization with expensive evaluations by combining the objectives through augmented Tchebycheff scalarization, fitting a Gaussian process surrogate, and selecting new evaluation points using expected improvement. HyperMapper 2.0 [20] handles design spaces with discrete variables and unknown feasibility constraints through separate regression models for objective values and classification models for feasibility. It has been applied to the automatic tuning of hardware accelerators expressed in the Spatial programming language. TuRBO [21] builds Gaussian process models within local trust regions, adapts the size of each region according to optimization progress, and allocates evaluation resources among multiple local searches. These methods provide a general foundation for surrogate-assisted optimization.

Hardware design studies further integrate these iterative optimization methods with domain-specific design variables and evaluation flows. Reagen et al. [22] use multiobjective

Bayesian optimization to explore the tradeoff between inference accuracy and energy consumption in deep neural network accelerators, treating numerical precision as a design variable. BOOM-Explorer [23] targets the RISC-V BOOM microarchitecture and combines active learning, deep-kernel Gaussian processes, and multiobjective Bayesian optimization. Its acquisition function is based on expected improvement in Pareto hypervolume. For systems integrating multiple accelerators, ARS-Flow [24] combines Pareto-region random resampling with a multiobjective genetic algorithm whose hyperparameters are controlled adaptively. Building on this framework, ARS-Flow 2.0 [25] uses particle swarm optimization to enhance Gaussian process regression and iteratively updates the surrogate through active learning and Pareto-region resampling. These studies link the selection of designs for ground-truth evaluation to the regions of the design space that are most relevant to optimization.

ApproxPilot-SE builds on this sequential optimization foundation by coupling measured-reference local calibration with surrogate updates during approximate-accelerator DSE. Each new measurement expands both the training set and the local reference set, supporting global model updates and differential delay correction for subsequent candidates. Table I compares representative approaches in terms of surrogate models, prediction targets, structural representations, calibration mechanisms, and search-time model updates.

III. ApproxPilot-SE Methodology

ApproxPilot-SE is an automated framework for approximate design optimization of domain-specific accelerators, as shown in Fig. 1. Given a target RTL design and an approximate arithmetic-unit library, the framework explores different arithmetic-unit configurations to improve hardware efficiency while maintaining application accuracy. Graph-based surrogate models are used to estimate the PPA and accuracy of candidate designs, while real evaluation results are used to update the models and guide subsequent exploration.

A. Problem Formulation

Approximate accelerator design aims to improve hardware efficiency while maintaining application accuracy. Given a target RTL design and an approximate arithmetic-unit library, the optimization task is to select a suitable implementation for each configurable arithmetic site. Different combinations of arithmetic units produce different trade-offs between PPA and accuracy.

Before constructing the design space, we prune the arithmetic-unit library using the strategies adopted in ApproxPilot [6]. Each unit is characterized by its arithmetic error, area, power, and latency. Dominated units are removed through multiobjective comparison. The remaining units form the candidate library for design optimization.

Let N be the number of configurable arithmetic sites. For site i, let e_i denote its exact implementation and $\mathcal{A}_i$ the set of compatible approximate implementations after pruning. The legal implementation set and the complete design space are defined as

$$\mathcal{C}_i = \{e_i\} \cup \mathcal{A}_i, \qquad \Omega = \prod_{i=1}^{N} \mathcal{C}_i. \tag{1}$$

A candidate design is represented by $\mathbf{x} = (x_1, \ldots, x_N) \in \Omega$, where x_i specifies the implementation selected for site i.

Each configuration has its own area $A(\mathbf{x})$, power $P(\mathbf{x})$, latency $L(\mathbf{x})$, and application accuracy $ACC(\mathbf{x})$. Let A_{ex}, P_{ex}, and L_{ex} denote the PPA metrics of the fully exact design. We formulate the optimization problem as

$$\min_{\mathbf{x}\in\Omega} \mathbf{f}(\mathbf{x}) = \left(\frac{A(\mathbf{x})}{A_{\text{ex}}}, \frac{P(\mathbf{x})}{P_{\text{ex}}}, \frac{L(\mathbf{x})}{L_{\text{ex}}}, 1 - \text{Acc}(\mathbf{x}) \right). \tag{2}$$

Here, larger values of $ACC(\mathbf{x})$ indicate higher accuracy. The first three objectives minimize normalized PPA, while the fourth maximizes accuracy. The goal is to find competitive configurations with different PPA and accuracy trade-offs.

B. Framework Overview

Fig. 1 presents the overall framework of ApproxPilot-SE. It consists of two stages: surrogate co-evolution and final DSE.

The framework takes the target RTL and the pruned arithmetic-unit library as inputs. It first constructs a configuration-aware graph from the circuit topology and arithmetic implementation choices. Each arithmetic operation is represented by a node, and edges describe the connections between operations. Different configurations share the circuit topology but have different node features.

Based on the graph representation, ApproxPilot-SE builds separate GNN-based surrogate models for PPA and accuracy prediction. The PPA model predicts area, power, and latency. The accuracy model estimates application accuracy. A path-aware readout is incorporated into the PPA model to support latency prediction. The model structures are presented in Section IV.

The co-evolution stage starts with an initial set of evaluated configurations. Their measured PPA and accuracy values are used to train the surrogate models. These configurations also provide references for local latency calibration.

During DSE, NSGA-III searches the design space using surrogate predictions. The models estimate the objectives of candidate configurations without requiring synthesis for every design. Local latency calibration adjusts the predicted latency using nearby measured configurations. The calibrated latency is combined with the predicted area, power, and accuracy to guide the search.

The search results are then used to select Pareto-relevant configurations for logic synthesis and accuracy evaluation. The resulting measurements are added to the training data, and both surrogate models are updated. Meanwhile, the measured configurations are added to the reference set for local latency calibration.

The updated models and references guide the next round of DSE. In this way, search and surrogate learning evolve together. DSE identifies configurations for real evaluation,

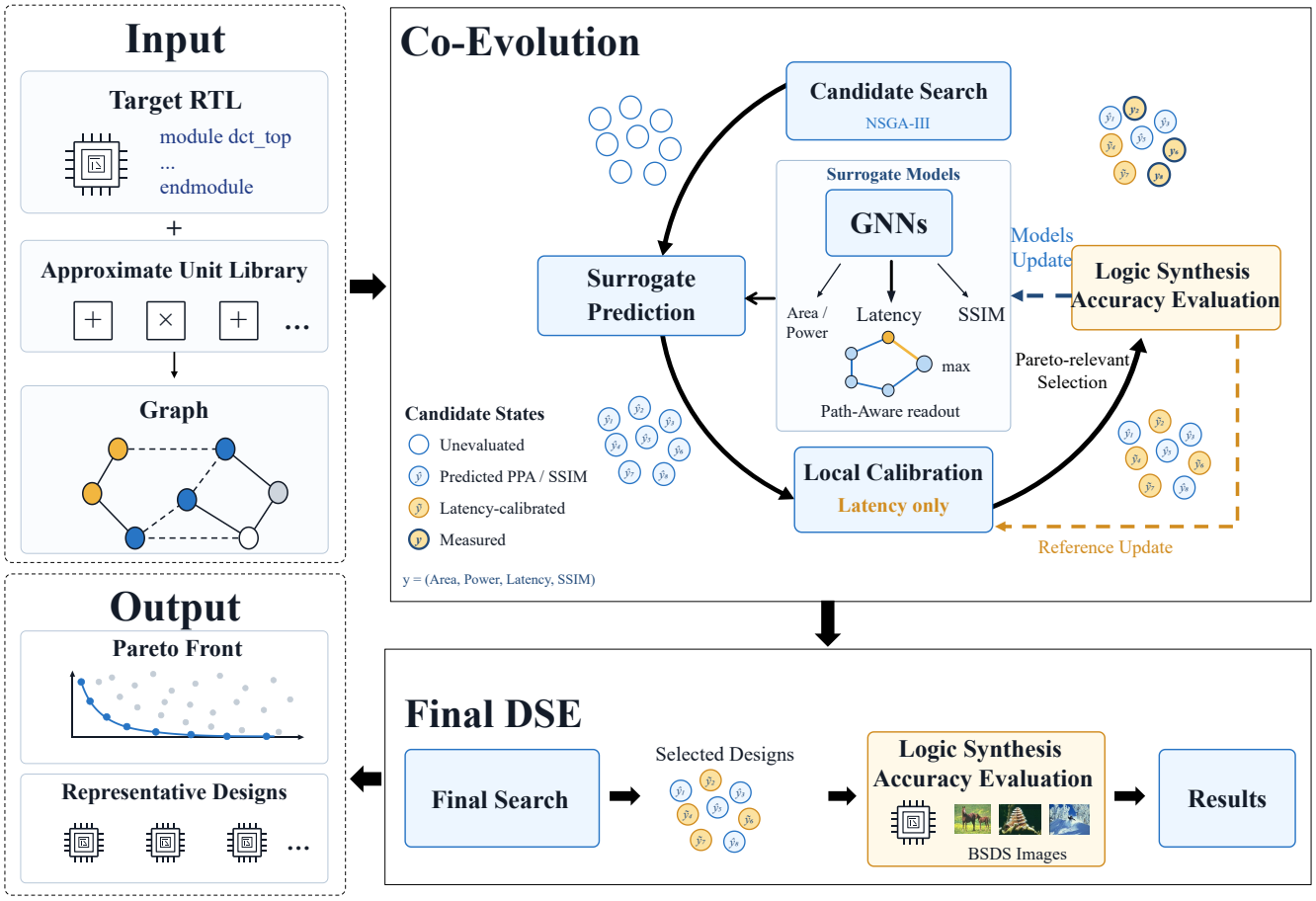


Fig. 1. Overview of ApproxPilot-SE. The framework combines surrogate prediction, local latency calibration, and iterative feedback. Measured results update the surrogate models and calibration references. Final DSE produces a Pareto front and representative designs based on measured PPA and accuracy.

while the new measurements improve the models used in subsequent search rounds.

After co-evolution, the updated surrogate models and calibration references are used for final DSE. NSGA-III searches the design space and selects promising configurations for logic synthesis and accuracy evaluation. The measured results are used to identify the Pareto front under the required accuracy constraint. Representative designs are then selected to provide different PPA and accuracy trade-offs.

IV. Surrogate Modeling

To efficiently evaluate accelerator configurations during DSE, ApproxPilot-SE employs two graph-based surrogate models. Given an accelerator configuration, the PPA model predicts its area, power, and latency, while the accuracy model estimates its application-level accuracy. Since latency depends on the paths formed by interconnected arithmetic operations, we introduce a path-aware readout and use nearby measured configurations to calibrate its predictions. During exploration, Pareto-relevant configurations are selected for real evaluation, and their measured results are used to update the surrogate models and calibration references. The surrogate models, latency calibration, and iterative feedback methods are detailed in the following subsections.

A. Surrogate Models

ApproxPilot-SE employs separate graph-based surrogates for PPA and application accuracy prediction. Following ApproxPilot [6], the target accelerator is represented as a graph in which arithmetic operations are nodes and their dependencies are edges. Different configurations share the accelerator topology, while their node features change according to the selected arithmetic units.

For a configuration $\mathbf{x}$, the two graph inputs are

$$G_k(\mathbf{x}) = (V, E_k, X_k(\mathbf{x})), \qquad k \in \{P, S\}, \tag{3}$$

where P and S denote the PPA and accuracy branches, respectively. The node set V represents arithmetic operations, E_k describes their dependencies, and $X_k(\mathbf{x})$ contains the configuration-dependent features.

The PPA surrogate uses a GNN to learn circuit representations from graph structure and node features. Separate readouts are used to predict area, power, and latency, with a path-aware readout designed for latency prediction. Its node features include operation type, bit width, signedness, graph depth, and connectivity information. Pre-characterized unit-level area, power, and latency are included together with their exact-unit references. Configuration-level features summarize the

TABLE II
REPRESENTATIVE FEATURES OF THE SURROGATE MODELS.

Feature Group	Description
Structure	Operation type, bit width, signedness, graph depth, and connectivity
Hardware	Unit-level area, power, latency, and exact-unit references
Approximation	Selected-unit identity and arithmetic error metrics
Configuration	Approximate-unit distribution and aggregate hardware features

distribution of approximate units and their aggregate hardware characteristics.

The accuracy surrogate uses a separate GNN to learn the relationship between arithmetic approximation and application accuracy. It predicts the SSIM of each evaluation image, and the average is used as the configuration-level accuracy. It uses the application graph and additional approximation features. These include unit identity and arithmetic error metrics such as mean absolute error (MAE), mean square error (MSE), error rate, and maximum absolute error. The error metrics describe the numerical behavior of the selected units, while the graph captures their positions in the application computation. Representative features are summarized in Table II.

Unlike area and power, latency depends strongly on the paths formed by connected arithmetic operations. We therefore introduce a path-aware readout and local calibration mechanism for latency prediction.

B. Latency Calibration

1) Local Calibration: ApproxPilot-SE further calibrates the predicted latency using previously measured configurations.

At feedback round t, let $\mathcal{R}_t$ denote the set of configurations with measured latency in the cumulative training set. For each reference $\mathbf{r} \in \mathcal{R}_t$, its prediction residual is

$$\rho_t(\mathbf{r}) = L(\mathbf{r}) - \widehat{L}_t(\mathbf{r}), \tag{4}$$

where $L(\mathbf{r})$ denotes the measured latency and $\widehat{L}_t(\mathbf{r})$ is the current surrogate prediction.

Configuration proximity is measured by Hamming distance:

$$d_H(\mathbf{x}, \mathbf{r}) = \sum_{i=1}^{N} \mathbb{I}[x_i \neq r_i], \tag{5}$$

where $\mathbb{I}[\cdot]$ is the indicator function and N is the number of configurable arithmetic sites.

For an unevaluated configuration $\mathbf{x}$, the nearest measured reference is

$$\mathbf{r}_t^*(\mathbf{x}) \in \arg\min_{\mathbf{r} \in \mathcal{R}_t} d_H(\mathbf{x}, \mathbf{r}). \tag{6}$$

Let $d_{\max}$ denote the maximum calibration distance. The calibrated latency estimate is

$$\widetilde{L}_t(\mathbf{x}) = \begin{cases} L_{\text{ex}}, & \mathbf{x} = \mathbf{x}_{\text{ex}}, \\ \widehat{L}_t(\mathbf{x}) + \rho_t(\mathbf{r}_t^*), & d_H(\mathbf{x}, \mathbf{r}_t^*) \leq d_{\max}, \\ \widehat{L}_t(\mathbf{x}), & \text{otherwise}. \end{cases} \tag{7}$$

The fully exact configuration retains its measured latency. If no reference lies within the calibration neighborhood, the direct surrogate prediction is used.

For a candidate with an applicable reference, Eq. 7 becomes

$$\widetilde{L}_t(\mathbf{x}) = L(\mathbf{r}_t^*) + \widehat{L}_t(\mathbf{x}) - \widehat{L}_t(\mathbf{r}_t^*). \tag{8}$$

The first term provides a measured latency anchor. The remaining terms estimate the latency difference between the candidate and the reference.

To examine the calibration effect, define the original prediction error as

$$e_t(\mathbf{x}) = \widehat{L}_t(\mathbf{x}) - L(\mathbf{x}). \tag{9}$$

Substituting Eq. 8 gives

$$e_{\text{cal},t}(\mathbf{x}) = e_t(\mathbf{x}) - e_t(\mathbf{r}_t^*). \tag{10}$$

Thus, the calibrated prediction error is the difference between the errors of the candidate and its reference. Suppose neighboring configurations share a common bias component b_t:

$$\begin{aligned} e_t(\mathbf{x}) &= b_t + \delta_t(\mathbf{x}), \\ e_t(\mathbf{r}_t^*) &= b_t + \delta_t(\mathbf{r}_t^*). \end{aligned} \tag{11}$$

The calibrated error then becomes

$$e_{\text{cal},t}(\mathbf{x}) = \delta_t(\mathbf{x}) - \delta_t(\mathbf{r}_t^*). \tag{12}$$

The shared bias is canceled, leaving the difference between the local error components. This provides the basis for using nearby measured configurations to correct latency estimates.

The calibration requires no additional learned correction model. It only modifies latency predictions.

2) Path-Aware Readout: To capture arithmetic dependencies in latency prediction, we apply a path-aware readout to the node representations produced by the PPA encoder. Inspired by max-plus propagation in timing analysis, the readout accumulates hidden representations along arithmetic dependencies in topological order:

$$\mathbf{s}_{v,d} = \begin{cases} \mathbf{h}_v, & \text{Pred}_d(v) = \varnothing, \\ \mathbf{h}_v + \max\limits_{u \in \text{Pred}_d(v)} \mathbf{s}_{u,d}, & \text{otherwise}, \end{cases} \tag{13}$$

where $\mathbf{h}_v$ is the representation of node v, d indexes an acyclic timing group $G_d = (V_d, E_d)$, and $\text{Pred}_d(v)$ denotes its predecessors within that group. The maximum is applied element-wise. The graph-level path representation is obtained as $\mathbf{z}_L = \max_d \max_{v \in V_d} \mathbf{s}_{v,d}$ and used for latency prediction without explicit critical-path labels.

C. Iterative Feedback

1) Feedback Selection: At round t, the current surrogates and latency calibration provide the predicted objective vector

$$\widehat{\mathbf{f}}_t(\mathbf{x}) = \left(\frac{\widehat{A}_t(\mathbf{x})}{A_{\text{ex}}}, \frac{\widehat{P}_t(\mathbf{x})}{P_{\text{ex}}}, \frac{\widetilde{L}_t(\mathbf{x})}{L_{\text{ex}}}, 1 - \widehat{\text{Acc}}_t(\mathbf{x}) \right). \tag{14}$$

NSGA-III uses these objectives to generate an archive of candidate configurations. Feedback selection is then performed

on the unique configurations that have not received real evaluation. Let $\mathcal{U}_t$ denote this candidate set.

ApproxPilot-SE combines Pareto-relevant candidate selection with accuracy-boundary exploration. The predicted accuracy retention of configuration $\mathbf{x}$ is

$$\widehat{r}_t(\mathbf{x}) = \frac{\widehat{\mathrm{Acc}_t}(\mathbf{x})}{\mathrm{Acc}_{\mathrm{ex}}}, \tag{15}$$

where $\mathrm{Acc}_{\mathrm{ex}}$ is the measured accuracy of the fully exact configuration.

Given an acquisition threshold τ_{acq}, the candidate set is partitioned into

$$\begin{aligned} \mathcal{U}_t^{+} &= \{\mathbf{x} \in \mathcal{U}_t \mid \widehat{r}_t(\mathbf{x}) \geq \tau_{\mathrm{acq}}\}, \\ \mathcal{U}_t^{-} &= \{\mathbf{x} \in \mathcal{U}_t \mid \widehat{r}_t(\mathbf{x}) < \tau_{\mathrm{acq}}\}. \end{aligned} \tag{16}$$

Main feedback candidates are selected from $\mathcal{U}_t^{+}$. They are ranked using nondominated sorting of the predicted objectives. Lower-ranked fronts receive priority. Within a partially selected front, crowding distance is used to preserve diversity.

Boundary candidates are selected from $\mathcal{U}_t^{-}$. Their proximity to the accuracy threshold is defined as

$$\Delta_t(\mathbf{x}) = \tau_{\mathrm{acq}} - \widehat{r}_t(\mathbf{x}), \qquad \mathbf{x} \in \mathcal{U}_t^{-}. \tag{17}$$

Candidates with smaller Δ_t are selected first, prioritizing configurations immediately below the predicted accuracy boundary.

Let b_t denote the feedback budget at round t, and let β be the maximum fraction allocated to boundary exploration. The candidate allocation is

$$\begin{aligned} b_t^{\mathrm{bd}} &= \min\left(\lfloor \beta b_t \rfloor, |\mathcal{U}_t^{-}|\right), \\ b_t^{\mathrm{main}} &= b_t - b_t^{\mathrm{bd}}. \end{aligned} \tag{18}$$

The main group receives the remaining feedback slots. When fewer boundary candidates are available, the unused boundary slots are assigned to the main group.

Let $\mathcal{S}_t^{\mathrm{main}}$ and $\mathcal{S}_t^{\mathrm{bd}}$ denote the selected main and boundary configurations, respectively. The feedback set is

$$\mathcal{S}_t = \mathcal{S}_t^{\mathrm{main}} \cup \mathcal{S}_t^{\mathrm{bd}}. \tag{19}$$

Both groups undergo system-level logic synthesis and application-level accuracy evaluation. Feedback acquisition is performed after surrogate-guided search and is distinct from NSGA-III population selection.

2) Model Updates: Let $\mathcal{D}_t$ denote the cumulative measured training set at round t. The selected configurations provide new measured labels

$$\mathcal{Y}_t = \{(\mathbf{x}, \mathbf{y}(\mathbf{x})) \mid \mathbf{x} \in \mathcal{S}_t\}, \tag{20}$$

where $\mathbf{y}(\mathbf{x})$ contains system-level PPA and the application accuracy measurements.

The training set is expanded as

$$\mathcal{D}_{t+1} = \mathcal{D}_t \cup \mathcal{Y}_t. \tag{21}$$

Both surrogate models are reinitialized and trained from scratch on the expanded training set. Let θ_t^P and θ_t^S denote the PPA and accuracy model parameters. The training procedure is expressed as

$$\begin{aligned} \theta_{t+1}^P &= \mathrm{Train}_P(\mathcal{D}_{t+1}; \mathrm{Init}()), \\ \theta_{t+1}^S &= \mathrm{Train}_S(\mathcal{D}_{t+1}; \mathrm{Init}()). \end{aligned} \tag{22}$$

Here, $\mathrm{Init}()$ denotes a fresh model initialization.

The feedback measurements also expand the latency calibration references:

$$\mathcal{R}_{t+1} = \mathcal{R}_t \cup \mathcal{S}_t. \tag{23}$$

After retraining, the reference residuals are recomputed using the updated latency surrogate:

$$\rho_{t+1}(\mathbf{r}) = L(\mathbf{r}) - \widehat{L}_{t+1}(\mathbf{r}), \qquad \mathbf{r} \in \mathcal{R}_{t+1}. \tag{24}$$

Thus, the same feedback measurements update both the surrogate models and the local calibration references. Algorithm 2 summarizes the procedure.

The updated surrogates and calibration references are used in subsequent search rounds. The models remain fixed during each search, and updates are performed only after the selected configurations have been evaluated.

Algorithm: Iterative Feedback

Require: Design space Ω, dataset $\mathcal{D}_0$, rounds T, budgets $\{b_t\}$, τ_{acq}, β, $d_{\max}$
Ensure: Updated surrogates and references
Train PPA and accuracy models on $\mathcal{D}_0$
Initialize measured references $\mathcal{R}_0$
for $t = 0, \ldots, T-1$ **do**
  Compute current reference residuals
  $\mathcal{U}_t \leftarrow \mathrm{Search}_{\mathrm{NSGAIII}}(\Omega)$
  Exclude evaluated and duplicate designs
  Partition $\mathcal{U}_t$ by τ_{acq}
  Select main candidates by Pareto rank and crowding distance
  Select boundary candidates nearest to τ_{acq}
  Evaluate selected configurations $\mathcal{S}_t$
  $\mathcal{D}_{t+1} \leftarrow \mathcal{D}_t \cup \mathcal{Y}_t$
  Reinitialize and retrain both surrogates
  $\mathcal{R}_{t+1} \leftarrow \mathcal{R}_t \cup \mathcal{S}_t$
end for
Refresh final calibration residuals
return Updated surrogates and references

Fig. 2. Iterative feedback selection and model updates.

V. Experimental Results

In this section, we evaluate ApproxPilot-SE for approximate accelerator design and compare it with ApproxPilot and ApproxGNN [6], [7]. We first examine the optimization results across different accelerators, including larger designs with substantially more configurable arithmetic sites. We then evaluate end-to-end data efficiency and investigate the effectiveness of iterative feedback, feedback selection, and latency modeling through ablation studies.

A. Experimental Setup

Benchmarks: We evaluate ApproxPilot-SE on five domain-specific accelerators: DCT, Laplace filtering, Sobel edge detection, Gaussian filtering, and KMeans clustering. Their arithmetic operations and configurable sites are summarized in Table IV.

TABLE III
VERIFIED PPA HV GAIN UNDER DIFFERENT SSIM-RETENTION CONSTRAINTS.RESULTS USE 720 TRAINING CONFIGURATIONS PER METHOD.

Benchmark	Method	$\geq 80\%$	$\geq 85\%$	$\geq 90\%$	$\geq 95\%$	$\geq 99\%$
DCT	ApproxPilot	0.035739	0.034385	0.023390	0.009963	0.000000
	ApproxGNN	0.025361	0.025361	0.020672	0.016977	0.000963
	ApproxPilot-SE	**0.101525**	**0.099683**	**0.088623**	**0.078963**	**0.064782**
Laplace	ApproxPilot	**0.422106**	0.000000	0.000000	0.000000	0.000000
	ApproxGNN	0.088416	0.088416	0.088416	0.088416	0.061240
	ApproxPilot-SE	0.381159	**0.332542**	**0.211086**	**0.144414**	**0.068019**
Sobel	ApproxPilot	0.208729	0.133068	0.081493	0.069325	0.008950
	ApproxGNN	**0.229965**	0.115541	0.054248	0.054248	0.009402
	ApproxPilot-SE	**0.229965**	**0.136205**	**0.096135**	**0.078954**	**0.021396**
Gaussian	ApproxPilot	0.290535	0.236497	0.186049	0.082716	0.000000
	ApproxGNN	0.295673	0.120928	0.055735	0.020681	0.004514
	ApproxPilot-SE	**0.329017**	**0.329017**	**0.243964**	**0.113170**	**0.045221**
KMeans	ApproxPilot	0.377168	0.019913	0.013244	0.005123	0.000746
	ApproxGNN	0.479862	0.066604	0.013502	0.013500	0.000000
	ApproxPilot-SE	**1.664181**	**0.663804**	**0.253267**	**0.117443**	**0.004112**

TABLE IV
BENCHMARK CONFIGURATIONS AND INITIAL DESIGN SPACES.

Benchmark	Operations	Sites	Initial Space
DCT	Add / Sub /Neg	76	3.26×10^{91}
Laplace	Add	60	1.77×10^{72}
Sobel	Add / Sub	5	2.04×10^{15}
Gaussian	Add / Mul	17	8.56×10^{26}
KMeans	Add / Sub / Mul / Sqrt	16	7.56×10^{34}

The DCT accelerator contains 80 arithmetic operations, including 76 configurable sites and four fixed exact operations. Laplace contains 60 configurable sites, while Sobel, Gaussian, and KMeans contain 5, 17, and 16 sites, respectively. The initial design spaces of DCT and Laplace reach approximately 3.26×10^{91} and 1.77×10^{72} configurations, respectively. Compared with the accelerators evaluated in the original ApproxPilot framework [6], these larger accelerator structures substantially expand the combinatorial design space and present greater challenges for system-level approximation optimization.

Approximate operator library: The arithmetic-unit library combines EvoApproxLib components and CGP-generated approximate units. Each configurable site is assigned compatible arithmetic implementations according to operation type, bit width, signedness, and interface requirements. The resulting accelerator configurations exhibit different hardware costs and numerical error characteristics, providing the design space for system-level approximation optimization.

Evaluation platform: Hardware area, power, and latency are obtained using Synopsys Design Compiler with a 45-nm technology library. Application quality is evaluated using the Structural Similarity Index Measure (SSIM) [26] on image-processing benchmarks, with evaluation images drawn from BSDS500 [27]. Accuracy retention is calculated relative to the fully exact implementation.

Framework settings: ApproxPilot-SE employs separate GNN models for PPA and application accuracy prediction. The PPA encoder contains four layers, while the accuracy encoder uses five layers. Both models have a hidden dimension of 128. NSGA-III is employed for multi-objective design space exploration, and local latency calibration uses a maximum Hamming distance of $d_{\max} = 10$.

ApproxPilot and ApproxGNN serve as the baseline methods, retaining their respective prediction models and search strategies. For DCT, we adopt a full training budget of 900 labeled configurations, with an additional 100 configurations reserved for validation. To evaluate data efficiency, we consider 60%, 80%, and 100% of the full training budget, corresponding to 540, 720, and 900 configurations, respectively. ApproxPilot-SE starts with 540 initial configurations and progressively expands its training set through search-selected feedback, adding 90 measured configurations per round. After optimization, each method selects 100 final candidates for independent ground-truth PPA and application-quality evaluation.

Evaluation metric: We employ three-dimensional PPA hypervolume (HV) gain to evaluate the optimization results under different SSIM-retention constraints. Area, power, and latency are normalized by the corresponding values of the exact accelerator. With reference point $\mathbf{r}$, HV gain is defined as

$$\begin{aligned} \mathrm{HV}_{\mathrm{Gain}} = \mathrm{HV}(\mathcal{P}_\tau \cup \{\mathbf{f}_{\mathrm{ex}}\}; \mathbf{r}) \\ - \mathrm{HV}(\{\mathbf{f}_{\mathrm{ex}}\}; \mathbf{r}), \end{aligned} \tag{25}$$

where $\mathbf{f}_{\mathrm{ex}} = (1, 1, 1)$ and $\mathcal{P}_\tau$ contains the measured PPA points satisfying the SSIM-retention threshold τ. A larger HV gain indicates better combined area, power, and latency tradeoffs beyond the exact implementation. All reported HV values are computed from ground-truth-verified designs.

B. Overall Approximation Optimization

We first compare ApproxPilot-SE with ApproxPilot and ApproxGNN across five accelerators under SSIM-retention requirements ranging from 80% to 99%. As shown in Table III, ApproxPilot-SE achieves the highest or equally highest

verified PPA HV Gain in 24 of the 25 evaluated cases, demonstrating its effectiveness across different accelerator structures and quality constraints.

The advantages are particularly evident on the larger DCT and Laplace benchmarks. At 90% SSIM retention, ApproxPilot-SE achieves 3.79× and 2.39× the HV Gain of the strongest baseline on DCT and Laplace, respectively. These results demonstrate the effectiveness of the proposed framework in exploring substantially larger combinatorial design spaces. Consistent improvements are also observed on Sobel, Gaussian, and KMeans, indicating its applicability across different arithmetic structures.

As the SSIM-retention requirement becomes more stringent, the feasible design space becomes increasingly restricted, making it more challenging to obtain favorable hardware tradeoffs. Nevertheless, ApproxPilot-SE maintains a substantial advantage in the high-accuracy region. On DCT, its improvement over the strongest baseline increases from 2.84× at 80% retention to 4.65× at 95% retention. Even at 99% retention, ApproxPilot-SE maintains a clear advantage and achieves the highest HV Gain on all five benchmarks. These results highlight its ability to identify hardware-efficient configurations while preserving high application accuracy.

Pareto Frontiers: Fig. 3 compares the Area–SSIM, Power–SSIM, and Delay–SSIM Pareto fronts obtained by ApproxPilot-SE, ApproxPilot, and ApproxGNN on DCT.

As shown in Fig. 3, ApproxPilot-SE achieves more favorable Pareto fronts across all three hardware metrics. Compared with the baseline methods, its solutions extend toward lower area, power, and delay over a broad range of SSIM levels. This improvement indicates that ApproxPilot-SE can identify a wider range of competitive hardware configurations rather than obtaining gains at only a few operating points. Consequently, designers can select different implementations according to their hardware constraints and application-quality requirements.

The advantage is particularly important in the high-SSIM region, where approximation opportunities are more limited. While aggressive approximation can achieve hardware savings at the expense of output quality, maintaining high SSIM requires more selective arithmetic-unit substitutions. ApproxPilot-SE continues to identify favorable PPA–quality tradeoffs in this region, demonstrating its ability to balance hardware efficiency and application accuracy. Together with the HV results in Table III, these observations show that ApproxPilot-SE improves both the overall quality of the explored design set and the available hardware tradeoffs under stringent accuracy requirements.

To further illustrate the practical benefits of these tradeoffs, Table V presents a representative DCT configuration identified by ApproxPilot-SE and compares its hardware performance and application quality with those of the exact implementation.

The optimized configuration retains 99.62% of the original SSIM while achieving reductions of 19.36%, 20.68%, and 9.70% in area, power, and delay, respectively. Notably, these improvements are achieved simultaneously, without sacrificing one hardware objective to improve another. This result demonstrates that ApproxPilot-SE can identify approximate configurations that preserve nearly the same application quality while delivering substantial hardware savings. It also provides a concrete example of how the favorable Pareto tradeoffs translate into improvements in a synthesized accelerator implementation.

TABLE V
REPRESENTATIVE OPTIMIZED DCT DESIGN.

Metric	Exact	ApproxPilot-SE	Reduction
Area (μm^2)	3498.166	2820.930	**19.36%**
Power (mW)	1.421650	1.127702	**20.68%**
Delay (ns)	2.797612	2.526248	**9.70%**
Mean SSIM	0.855541	0.852312	–
SSIM retention	100%	99.62%	–

End-to-End Data Efficiency: While the preceding results demonstrate the optimization quality of ApproxPilot-SE, the efficiency of obtaining these results is equally important. System-level synthesis and application-quality evaluation are computationally expensive, making the effective utilization of measured training data essential for large-scale approximate accelerator optimization. To investigate this aspect, Fig. 4 compares the three frameworks using 60%, 80%, and 100% of the full training budget, corresponding to 540, 720, and 900 configurations, respectively.

As shown in Fig. 4, ApproxPilot-SE consistently achieves substantially higher verified HV Gain than both baselines across all three training budgets. Remarkably, using only 60% of the available training labels, ApproxPilot-SE achieves 3.42× the HV Gain of the strongest baseline using the full training budget. This result highlights the ability of ApproxPilot-SE to identify competitive accelerator configurations with considerably fewer ground-truth measurements, rather than relying on larger training datasets to obtain superior optimization results.

The advantage is also maintained under equal training budgets, reaching 3.79× and 3.89× the strongest baseline at 720 and 900 training labels, respectively. Moreover, increasing the training budget from 540 to 720 improves the HV Gain of ApproxPilot-SE by 11.2%, whereas a further increase to 900 provides an additional 2.1% improvement. This trend indicates that substantial optimization benefits can already be obtained with a reduced training budget, with progressively smaller gains from additional measurements over the evaluated range. By incorporating search-selected configurations into subsequent surrogate updates, ApproxPilot-SE makes effective use of the available measurements to guide the exploration process.

Beyond measurement efficiency, we also compare the overall optimization time. With 720 training configurations, the estimated end-to-end runtimes of ApproxPilot-SE, ApproxPilot, and ApproxGNN are 4.47–6.00 h, 4.08–5.74 h, and 5.65–8.07 h, respectively. ApproxPilot-SE exhibits a runtime range comparable to ApproxPilot and generally lower than ApproxGNN, while achieving significantly higher verified HV Gain. These results demonstrate that ApproxPilot-SE improves the quality of the final optimized designs without requiring a substantial increase in overall optimization time.

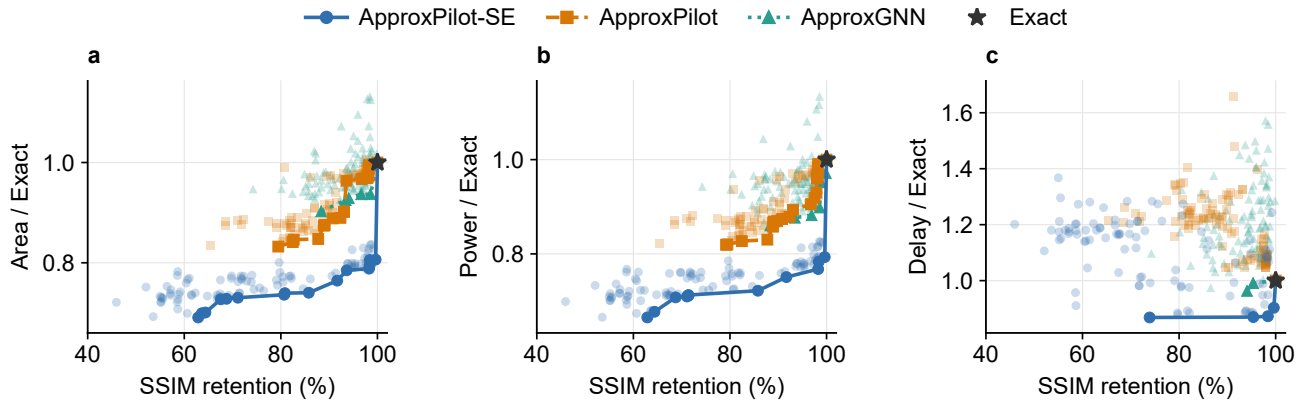


Fig. 3. Measured DCT PPA–SSIM tradeoffs: (a) Area, (b) Power, and (c) Delay.

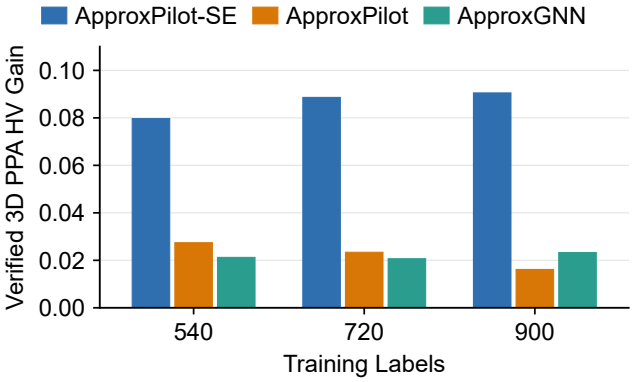


Fig. 4. End-to-end data efficiency on DCT under 90% SSIM retention. HV Gain is computed from ground-truth-verified final designs.

TABLE VI
END-TO-END ABLATION RESULTS ON DCT. IMPROVEMENT DENOTES THE GAIN OF THE FULL METHOD OVER EACH VARIANT.

Modification	HV Gain	Improvement
Full ApproxPilot-SE	**0.088623**	–
Replace iterative feedback with one-shot training	0.078411	+13.02%
Replace guided selection with random selection	0.076516	+15.82%
Remove local Delay calibration	0.073336	+20.85%
Replace path-aware readout with global-max pooling	0.080654	+9.88%
Replace the Delay model with ApproxPilot's two-stage model	0.078382	+13.07%

C. Ablation Studies

We conduct end-to-end ablation studies on DCT to evaluate the contributions of iterative feedback, feedback selection, and latency modeling. All variants use 720 training configurations, with 100 final candidates evaluated using ground-truth PPA and SSIM. Table VI reports the verified HV Gain under 90% SSIM retention. The comparisons focus on the impact of each technique on the final optimization results.

Iterative Feedback and Selection: As shown in Table VI, ApproxPilot-SE achieves 13.02% higher verified HV Gain than one-shot optimization under the same training budget. This improvement confirms the benefit of integrating surrogate updates into DSE rather than separating model training from the search process. Measurements acquired during exploration allow subsequent surrogate models to reflect the design regions encountered by the search, improving the final optimization outcome.

The selection of feedback configurations is equally important. Replacing the proposed selection strategy with random acquisition reduces the optimization performance, with ApproxPilot-SE achieving a 15.82% higher HV Gain. This comparison shows that the benefit of feedback depends not only on updating the models, but also on selecting configurations relevant to the optimization objectives. Together, these results establish the effectiveness of the proposed co-evolution between surrogate learning and design space exploration.

TABLE VII
SENSITIVITY OF DCT OPTIMIZATION TO FEEDBACK PARAMETERS.

(a) Boundary ratio

Ratio	Mean HV Gain
0%	0.090591
20%	0.089695
40%	0.092404

(b) Feedback schedule

Schedule	Mean HV Gain
45×4	0.084799
90×2	0.089695
180×1	0.087320

Feedback Parameter Sensitivity: We further examine whether the optimization performance depends strongly on the feedback settings. Table VII compares different accuracy-boundary ratios and feedback schedules. Each result is the mean verified HV Gain from three runs.

As shown in Table VII, ApproxPilot-SE maintains comparable optimization performance across different feedback settings. Varying the boundary ratio from 0% to 40% results in a maximum difference of approximately 3% in mean HV Gain. Similarly, the three feedback schedules produce results within approximately 6% of each other under the same measurement budget.

These results indicate that ApproxPilot-SE exhibits limited sensitivity to the evaluated feedback parameters. We therefore

adopt the intermediate settings of a 20% boundary ratio and two feedback rounds of 90 configurations each. The consistently competitive results across different settings demonstrate that the effectiveness of the proposed framework does not rely on precise parameter tuning.

Latency Modeling: The remaining ablations examine how latency modeling affects the final PPA optimization. Among the evaluated component removals, disabling local Delay calibration causes the largest reduction in verified HV Gain. The complete framework achieves a 20.85% improvement over the uncalibrated variant, demonstrating the practical value of incorporating measured reference configurations into latency estimation. This improvement also shows that local calibration benefits the actual selection of accelerator designs, beyond its effect on prediction accuracy.

Path-aware readout provides a further improvement of 9.88% over conventional global-max pooling. The result supports incorporating computation-path dependencies into the latency representation, rather than relying solely on graph-level pooling. Furthermore, replacing the proposed Delay model with the two-stage architecture of ApproxPilot reduces the final HV Gain, with the complete framework achieving a 13.07% advantage. These comparisons demonstrate that both local calibration and path-aware latency modeling contribute to identifying more competitive hardware configurations.

Overall, the ablation studies confirm that the optimization gains of ApproxPilot-SE arise from the combination of iterative surrogate updates, guided measurement selection, and hardware-aware latency estimation. Their contributions are reflected directly in verified system-level optimization results.

VI. Conclusion

In this paper, we proposed ApproxPilot-SE, an automated approximate accelerator optimization framework. The framework employs configuration-aware GNNs to predict hardware PPA and application quality, incorporating path-aware aggregation and measured-reference local calibration for Delay estimation. By integrating search-guided measurements into iterative surrogate updates, ApproxPilot-SE enables the surrogate models and DSE process to evolve together, improving the utilization of available synthesis data.

Experiments on five domain-specific accelerators, including DCT and Laplace with substantially larger combinatorial design spaces, demonstrate the effectiveness of ApproxPilot-SE. On DCT, the proposed framework achieves 3.79 times PPA HV Gain of the strongest baseline under the same training budget at 90% SSIM retention. Notably, even with 40% fewer training labels, it achieves 3.42 times the gain of the strongest baseline using the full budget. ApproxPilot-SE also maintains superior optimization performance under stringent accuracy constraints, achieving the highest HV Gain across all five benchmarks at 99% SSIM retention.

A representative DCT design preserves 99.62% of the original SSIM while reducing area, power, and delay by 19.36%, 20.68%, and 9.70%, respectively. End-to-end ablation studies further validate the effectiveness of iterative feedback, guided candidate selection, and latency modeling in improving the final optimization results. Together, these findings demonstrate the capability of ApproxPilot-SE to efficiently explore large approximate accelerator design spaces and achieve superior PPA–accuracy tradeoffs across different applications.

References

[1] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proceedings of the IEEE*, vol. 108, no. 12, pp. 2108–2135, 2020.

[2] H.-H. Que, Y. Jin, T. Wang, M.-K. Liu, X.-H. Yang, and F. Qiao, "A survey of approximate computing: From arithmetic units design to high-level applications," *Journal of Computer Science and Technology*, vol. 38, no. 2, pp. 251–272, 2023.

[3] Q. Xu, T. Mytkowicz, and N. S. Kim, "Approximate computing: A survey," *IEEE Design & Test*, vol. 33, no. 1, pp. 8–22, 2016.

[4] V. Mrazek, R. Hrbacek, Z. Vasicek, and L. Sekanina, "EvoApprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods," in *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition (DATE)*, 2017, pp. 258–261.

[5] V. Mrazek, M. A. Hanif, Z. Vasicek, L. Sekanina, and M. Shafique, "autoAx: An automatic design space exploration and circuit building methodology utilizing libraries of approximate components," in *Proceedings of the 56th Annual Design Automation Conference (DAC)*, 2019, pp. 1–6.

[6] Q. Zhang, S. Liu, Y. Hui, and C. Liu, "ApproxPilot: A GNN-based accelerator approximation framework," in *Proceedings of the International Symposium of Electronics Design Automation (ISEDA)*, 2025, pp. 226–231.

[7] O. Vlcek and V. Mrazek, "ApproxGNN: A pretrained GNN for parameter prediction in design space exploration for approximate computing," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025, pp. 1–8.

[8] S. Ullah, S. S. Murthy, and A. Kumar, "SMApproxLib: Library of FPGA-based approximate multipliers," in *Proceedings of the 55th Annual Design Automation Conference (DAC)*, 2018, pp. 1–6.

[9] K. Nepal, Y. Li, R. I. Bahar, and S. Reda, "ABACUS: A technique for automated behavioral synthesis of approximate computing circuits," in *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition (DATE)*, 2014, pp. 1–6.

[10] J. Castro-Godinez, J. Mateus-Vargas, M. Shafique, and J. Henkel, "AxHLS: Design space exploration and high-level synthesis of approximate accelerators using approximate functional units and analytical models," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2020, pp. 1–9.

[11] B. S. Prabakaran, V. Mrazek, Z. Vasicek, L. Sekanina, and M. Shafique, "ApproxFPGAs: Embracing ASIC-based approximate arithmetic components for FPGA-based systems," in *Proceedings of the 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.

[12] ——, "Xel-FPGAs: An end-to-end automated exploration framework for approximate accelerators in FPGA-based systems," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023, pp. 1–9.

[13] N. Wu, H. Yang, Y. Xie, P. Li, and C. Hao, "High-level synthesis performance prediction using GNNs: Benchmarking, modeling, and advancing," in *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, 2022, pp. 49–54.

[14] C. Xu, C. Kjellqvist, and L. W. Wills, "SNS's not a synthesizer: A deep-learning-based synthesis predictor," in *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, 2022, pp. 847–859.

[15] Z. Lin, Z. Yuan, J. Zhao, W. Zhang, H. Wang, and Y. Tian, "PowerGear: Early-stage power estimation in FPGA HLS via heterogeneous edge-centric GNNs," in *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition (DATE)*, 2022, pp. 1341–1346.

[16] E. Ustun, C. Deng, D. Pal, Z. Li, and Z. Zhang, "Accurate operation delay prediction for FPGA HLS using graph neural networks," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2020, pp. 1–9.

[17] Z. Guo, M. Liu, J. Gu, S. Zhang, D. Z. Pan, and Y. Lin, "A timing engine inspired graph neural network model for pre-routing slack prediction," in *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, 2022, pp. 1207–1212.

[18] H. Mo, Y. Wu, H. Jiang, Z. Ma, F. Lombardi, J. Han, and L. Liu, "Learning the error features of approximate multipliers for neural network applications," *IEEE Transactions on Computers*, vol. 73, no. 3, pp. 842–856, 2024.
[19] J. Knowles, "ParEGO: A hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 1, pp. 50–66, 2006.
[20] L. Nardi, D. Koeplinger, and K. Olukotun, "Practical design space exploration," in *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2019, pp. 347–358.
[21] D. Eriksson, M. Pearce, J. Gardner, R. D. Turner, and M. Poloczek, "Scalable global optimization via local bayesian optimization," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
[22] B. Reagen, J. M. Hernandez-Lobato, R. Adolf, M. A. Gelbart, P. N. Whatmough, G.-Y. Wei, and D. M. Brooks, "A case for efficient accelerator design space exploration via bayesian optimization," in *Proceedings of the IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, 2017, pp. 1–6.
[23] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. F. Wong, "BOOM-Explorer: RISC-V BOOM microarchitecture design space exploration framework," in *2021 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2021, pp. 1–9.
[24] S. Huang, Y. Ye, H. Yan, and L. Shi, "ARS-Flow: A design space exploration flow for accelerator-rich system based on active learning," in *Proceedings of the 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024, pp. 213–218.
[25] ——, "ARS-Flow 2.0: An enhanced design space exploration flow for accelerator-rich system based on active learning," *Integration*, vol. 101, p. 102315, 2025.
[26] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
[27] P. Arbeláez, M. Maire, C. Fowlkes, and J. Malik, "Contour detection and hierarchical image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 5, pp. 898–916, 2011.